CSCE553-001-202720 Prin of Software Methodology

M2 - Inclass Prep

This is a preparation for the M2 assignment. You will learn to use the various tools.

By the end of this session you should be able to:

1. Name the **path** and the **load** you are about to apply.
2. Drive your M1 API with `curl` (GET, POST, PATCH) including login.
3. Time a request with `curl -w` and read the pieces of that time.
4. Put a JWT (or session cookie) in an environment variable and reuse it.
5. Open Render Metrics and the OS monitor (Task Manager, Activity Monitor, or `top`), and record a **timestamped** CPU/memory sample plus a `pg_stat_activity` count.

Keep a notes file (paper, a doc, or `m2/results/prep-notes.md` in your app repo). Fill every **Write down** box. Those answers become the method section of M2.

Use **your** M1 system. Do not point anything at a classmate's URL.

---

0. Bring to class

Need	Why
Public HTTPS URL from M1	Hosted checks
Demo email + password (from your README)	Login
API table from your README	Map paths
A terminal (PowerShell, Terminal, or VS Code / Cursor)	Commands
Render (or your host) and Supabase (or your DB) dashboards open	Monitoring
Python 3.11+	Optional: `login.py` and a tiny driver smoke

You do **not** need Locust, k6, or `hey` today.

Load rules for this hour: one request at a time is the default. A handful of sequential `curl`s is fine. Do **not** start a 60-second flood, Locust, or `hey -n 2000`. That is M2 lab work, on **your** copy only.

Three different things. Mixing them up makes M2 graphs unreadable.

1.1 Path — *which work the server does*

A **path** is an HTTP method + URL that exercises a particular slice of your system. Same user, same token; different code and different database work.

Path name (M2)	What it is for	Typical call (adjust to your README)
Control	Cheap liveness. Almost no DB.	`GET /health`
Read	List or search. The main saturation sweep later.	Social: `GET /api/messages`. E-commerce: `GET /api/products` or `GET /api/products?q=`
Write	Create or update. Touches more DB (insert/update, maybe indexes).	Social: `POST /api/messages`. E-commerce: `POST /api/cart/items` or `PATCH` quantity
Mix	Some reads, some writes in one run	80% read path + 20% write path (the Python driver does this; not `curl` today)

HTML pages (`GET /`, `GET /feed`) used in the browser are **not** paths. They involve loading templates and static files. In this exercise we are not measuring user experience from interaction. Hence, use the `/api/...` (and `/health`).

If you system is not architected to have a clean API and UI separation, please rearchitect your system or talk to the instructor about it.

Write down — your paths

Path name	Method	Full URL (origin + path)	Auth required?	One-line: what the server does
Control	GET			
Read				
Write				

If your README uses different names, that is fine. The **role** (control / read / write) must still be clear.

1.2 Load — *how hard you push that path*

A **load** is the independent variable you set. The path stays the same; the push changes.

Offered concurrency N	How many clients are in flight at once	Stages 1, 4, 8, 16 (32 only if still healthy)
Duration	How long that intensity lasts	10 s warmup (discard) + 60 s measure
Mix	Fraction of requests that are reads	100% read, or 80/20
Payload	Bytes in a typical response	Hold constant; note it from `curl` today

Closed-loop concurrency (*N* clients, each sending the next request when the last returns) is **not** "N people on the website." Converting user counts to RPS is the worksheet in the M2 assignment — not this hour.

Today you will apply **tiny** load: *N* = 1, a few seconds, one path at a time.

1.3 Environment — *where the API process runs*

Series	API runs	Database	What it isolates
H — Hosted	Your public URL (Render or equivalent)	Same Postgres as M1	What graders hit; sleep, 0.1 CPU, extra RTT
L — Local API	`localhost` (same commit)	Prefer the **same** hosted Postgres	Hosting vs your laptop

Today: do the `curl` work on **H** first (the URL graders use). If time remains, repeat health + one read on **L**.

Write down

```
BASE_H = https://_________________.onrender.com     (no trailing slash)
BASE_L = http://127.0.0.1:8000                     (if you run local)
Git commit you are on: _________________
```

2. `curl` on your machine

`curl` is a command-line HTTP client. Graders, the M2 driver, and you all speak HTTP. The browser is optional for this lab.

2.1 Make sure you have the real `curl`

Windows (PowerShell): `curl` is often an **alias** for `Invoke-WebRequest`. That is a different program. The flags in this handout (`-w`, `-o`, `-H`) are for **curl.exe**.

```
Get-Command curl
curl.exe --version
```

If `Get-Command curl` says `Alias`, use **`curl.exe`** for every command below. If `curl.exe --version` fails, install **curl** or use Git Bash (Git for Windows includes curl).

macOS / Linux:

```
curl --version
```

Below, examples use `curl`. On Windows PowerShell, type `curl.exe` wherever you see `curl`.

2.2 Anatomy of a request

```
curl [options] URL
```

Options	Meaning
`-s`	Silent: no progress bar
`-S`	Still show errors if `-s` is on. Prefer **`-sS`** together
`-X METHOD`	HTTP method (`GET` is the default)
`-H "Name: value"`	Request header (repeat `-H` for several)
`-d '...'`	Request body (implies POST unless you also set `-X`)
`-o file`	Write **body** to a file (`NUL` on Windows, `/dev/null` on macOS/Linux to discard)
`-w "format"`	After the transfer, print extra facts (status, times, sizes)
`-v`	Verbose: headers in and out. Use when you are stuck; not for timing
`-i`	Include response **headers** in the output

Set the origin once so you do not retype the host.

PowerShell:

```
$env:BASE = "https://YOUR.onrender.com"    # no trailing slash
curl.exe -sS "$env:BASE/health"
```

macOS / Linux / Git Bash:

```
export BASE=https://YOUR.onrender.com
curl -sS "$BASE/health"
```

If the first hosted request hangs for ~30–60 s, the free web service is waking up. Wait. That wait is **cold start**, not a broken app. Retry until `/health` returns in well under a second. Write the slow first number down; you will need a cold-start measurement in M2.

Write down: first `/health` wall time (slow or fast): _____. After warmup, `/health` feels: _____.

2.3 HTTP methods you will actually use

Method	Typical use in this course	Body?	Example path
GET	Read. Should not create rows.	No (query string is OK)	`/health`, `/api/messages`, `/api/products?q=mug`
POST	Create, or "do this" (login, like, checkout)	Usually JSON	`/api/auth/login`, `/api/messages`
PATCH	Partial update	JSON	`/api/messages/{id}`, cart quantity
PUT	Replace a resource (some APIs)	JSON	Only if your README says so
DELETE	Remove (not required for M2)	Rarely	Only if you need it

Login is **POST** even though you "get" a token: you are sending a password and creating a session.

JSON bodies need a header:

```
-H "Content-Type: application/json"
```

Quoting: macOS/Linux can use single quotes around JSON. PowerShell is happier with escaped double quotes inside `-d`.

PowerShell:

```
  -H "Content-Type: application/json"
  -d "{\"email\":\"alex@example.com\",\"password\":\"ClassDemo123!\"}"
```

macOS / Linux:

```
curl -sS -X POST "$BASE/api/auth/login" \
  -H "Content-Type: application/json" \
  -d '{"email":"alex@example.com","password":"ClassDemo123!"}'
```

Replace email/password with **your** demo user. The backtick `` ` `` (PowerShell) and backslash `\` (bash) continue the command on the next line.

Do this now

1. `GET /health` — expect 200 and a small JSON body.
2. `POST` login — expect 200 and a token field (see §4).
3. If login is 404, your path is not `/api/auth/login`. Use the README. Write the real path in the table in §1.1.

Pretty-print JSON without extra tools:

```
curl -sS "$BASE/health" | python -m json.tool
```

3. Timing and size: `curl -w`

M2 cares about **latency pieces** and **payload size**, not only "it felt slow." `curl` can print both. Times are in **seconds** (so `0.250` is 250 ms).

%{variable}	What it measures
http_code	Status (200, 401, 502, ...). Empty body with 401 is still a completed HTTP call
time_namelookup	DNS. High on first call, often ~0 later
time_connect	TCP connect (includes DNS). Jump here often means network or the host is down
time_appconnect	TLS handshake (HTTPS only). 0 on plain http://127.0.0.1
time_pretransfer	Up to just before curl sends the request body / finishes request setup
time_starttransfer	Time to **first byte** (TTFB): network + server work before the body starts
time_total	Whole transfer, including reading the body. Closest to "how long the client waited"
size_download	Response **body** bytes. This is your payload note for M2
size_header	Response header bytes
num_connects	New connections used. 1 is normal for a single curl
url_effective	Final URL after redirects
errormsg	Transfer error text (timeouts, DNS fail). Empty on HTTP 4xx/5xx — those still have an http_code
redirect_url	If you were bounced (login page HTML instead of JSON — wrong path)

Rule of thumb: **server-side work** sits mostly in `time_starttransfer − time_pretransfer` (the wait after the request is sent, before the first byte). **Download** of a large body sits in `time_total − time_starttransfer`. Cold start inflates **all** of these, especially TTFB.

3.2 A reusable format

Discard the body so the terminal shows only the metrics. **Windows:** `-o NUL`. **macOS/Linux:** `-o /dev/null`.

PowerShell (hosted health):

```
curl.exe -sS -o NUL -w "http_code=%{http_code}`ntime_namelookup=%{time_namelookup}`ntime_connect=%{time_connect}`ntime_appconnect=%{time_appconnect}`ntime_starttransfer=%{time_starttransfer}`ntime_total=%{time_total}`nsize_download=%{size_download}`n" "$env:BASE/health"
```

macOS / Linux:

```
time_connect=%{time_connect}
time_appconnect=%{time_appconnect}
time_starttransfer=%{time_starttransfer}
time_total=%{time_total}
size_download=%{size_download}
" "$BASE/health"
```

One-liner used in M2 for cold start (total time only):

```
curl -sS -o NUL -w "cold_ms=%{time_total}\n" https://YOUR.onrender.com/health
```

(`NUL` → `/dev/null` on macOS/Linux.) `time_total` is still **seconds**. Multiply by 1000 for milliseconds in the report.

Do this now — three timings on H, after warmup

Call	`http_code`	`time_total` (s)	`time_starttransfer` (s)	`size_download` (bytes)
`GET /health`				
Authenticated **read** path				
Authenticated **write** path				

Fill the read/write rows after you have a token (§4–5). If write is much slower than health, that is expected: more code, more Postgres.

Write down: payload of the **read** path ≈ _____ bytes. Hold that endpoint constant in M2 while you vary concurrency.

---

4. Authentication and exporting a JWT

Load tests cannot click “Sign in with Google.” You need **email/password over the API**, then a credential on every later request.

Typical M1 JSON (FastAPI / build-guide style):

```
{"access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
 "token_type": "bearer"}
```

Some apps use `"token"` instead of `"access_token"`. Cookie-session apps may return `Set-Cookie` and an empty or `{ "ok": true }` body — see §4.4.

A JWT is three base64 segments separated by dots (`header.payload.signature`). Treat it like a **password**: it proves who you are until it expires. Never commit it, never paste it into a public chat, never put it in a screenshot of your report.

4.2 Login, then export `M2_TOKEN`

Option A — `login.py` (from the M2 handout pack; copy into your app repo). It prints the token field if the JSON looks like M1:

```
python login.py https://YOUR.onrender.com alex@example.com ClassDemo123!
```

If it prints a whole Python dict, your JSON uses another key. Open the printed body, find the token, and either edit `login.py` or use Option B.

PowerShell (capture the printout):

```
$env:M2_TOKEN = python login.py $env:BASE alex@example.com ClassDemo123!
$env:M2_TOKEN.Substring(0, [Math]::Min(20, $env:M2_TOKEN.Length))   # peek; should look like eyJ...
```

macOS / Linux:

```
export M2_TOKEN="$(python login.py "$BASE" alex@example.com ClassDemo123!)"
echo "${M2_TOKEN:0:20}..."
```

Option B — `curl` + Python (no extra packages). Use **your** login path and demo user.

PowerShell:

```
$raw = curl.exe -sS -X POST "$env:BASE/api/auth/login" `
  -H "Content-Type: application/json" `
  -d "{\"email\":\"alex@example.com\",\"password\":\"ClassDemo123!\"}"
$raw
$env:M2_TOKEN = ($raw | python -c "import sys,json; d=json.load(sys.stdin); print(d.get('access_token') or
d.get('token') or '')")
```

macOS / Linux:

```
export M2_TOKEN="$(
  curl -sS -X POST "$BASE/api/auth/login" \
    -H "Content-Type: application/json" \
    -d '{"email":"alex@example.com","password":"ClassDemo123!"}' \
  | python -c "import sys,json; d=json.load(sys.stdin); print(d.get('access_token') or d.get('token') or '')"
)"
echo "${M2_TOKEN:0:20}..."
```

If `M2_TOKEN` is empty, print `$raw` / the curl output. Common causes: wrong path, email confirmation still on, 401 bad password, HTML error page (you hit the UI route, not `/api/...`).

Check the variable is only in this terminal. Close the window and it is gone. That is what you want. Do not put the token in `.env` files you might commit. The M2 driver reads `--token` or the environment variable `M2_TOKEN`.

PowerShell reuse later:

macOS / Linux:

```
curl -sS "$BASE/api/messages" -H "Authorization: Bearer $M2_TOKEN"
```

E-commerce: swap in `/api/products` (or whatever your read path is).

4.3 Prove auth is doing something

1. **With** the header: read path → 200 and JSON data.
2. **Without** the header: same URL → **401** (or 403). If you still get 200, that endpoint is not protected; pick a path that requires login for the M2 read sweep, or you will not be testing the authenticated app.
3. Change one character of the token → 401.

Write down: login path _____. JSON key for the token _____. Token lifetime if you know it (code or docs) _____. What happens at expiry during a long M2 run: login again, export a new `M2_TOKEN`, note it as a confound.

4.4 If you used cookies instead of JWT

Some apps set a session cookie on login. Then:

```
curl -sS -c cookies.txt -X POST "$BASE/api/auth/login" ...
curl -sS -b cookies.txt "$BASE/api/messages"
```

`Bearer …`. If you only have cookies, you will need a small driver change later — tell the instructor this hour if that is your M1 design.

5. Exercise each path (still *N* = 1)

Use the token from §4. Replace paths with **your** table from §1.1.

5.1 Control (no auth)

```
curl -sS -o NUL -w "http_code=%{http_code} total_s=%{time_total} bytes=%{size_download}\n" "$BASE/health"
```

5.2 Read (auth)

Social example:

```
curl -sS -H "Authorization: Bearer $M2_TOKEN" "$BASE/api/messages"
```

Then the same URL with `-o` and `-w` so you get `size_download` without dumping a huge feed to the screen.

If the body is `[]` or only a couple of rows, the later read sweep will measure an empty table, not a product. Before M2, copy `seed_data.py` and POST ~50 rows through the **same create API** the read path lists (social: posts; e-commerce: **products**, not the cart). Check `size_download` again and write that number down. Do not treat seeding as a load test.

Social example:

```
curl -sS -X POST "$BASE/api/messages" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $M2_TOKEN" \
  -d "{\"body\":\"prep exercise post\"}"
```

E-commerce example (adjust fields to your README):

```
curl -sS -X POST "$BASE/api/cart/items" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $M2_TOKEN" \
  -d "{\"product_id\":1,\"quantity\":1}"
```

Expect **201** or **200** and a JSON object. **400/422** means the body does not match your API — copy a working example from your M1 README rather than guessing.

5.4 PATCH (if you have an update)

```
curl -sS -X PATCH "$BASE/api/messages/ID" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $M2_TOKEN" \
  -d "{\"body\":\"edited in prep\"}"
```

Put a real `ID` from the POST response or from the list.

hardest, and why you think that (code, not vibes).

6. Resource signals — API CPU and memory

M2 needs a **chain of evidence**: load → service numbers → **resource** numbers, on the **same clock**. "The API got slow" is not a claim until a resource moved with it.

Use **one machine's clock** (your laptop). Screenshot so a clock or the filename carries the time: `render-cpu-H-prep-154735.png`.

6.0 What to record (gauges, not a movie)

Task Manager, Activity Monitor, Resource Monitor, `top`, and `docker stats` are **live gauges** (about a one-second average). They are not lab recorders. You do not transcribe the needle every second. You write a **timestamped sample**.

One line is enough:

```
15:47:35  L-read-c8  mid-stage  process=python/uvicorn  CPU=72%  RSS=180 MB
```

For the stage you will argue from in M2 (today: idle vs one curl), take **three** samples when you can:

1. **Idle** — a few seconds with no load.
2. **Mid-work** — while the request (later: the 60 s window) is running.
3. **After recover** — 30–60 s later: did CPU/memory come back down?

If the value jumps, record a **range** (`CPU 60–80%`) and still keep the clock. Instantaneous is fine. "It looked high" is not.

Which box you are measuring

Series	API actually runs	Watch this	Do not use this for API CPU
H — Hosted	Render (or your host)	Host **Metrics** graph (§6.1)	Task Manager / Activity Monitor / `top` on the laptop (that is mostly `curl` or the Python driver)
L — Local	`localhost` on your laptop	Task Manager, Activity Monitor, `top`, or `docker stats` (§6.3)	Render Metrics (the hosted process is not in this run)

Laptop on **AC power** for any local timing you care about.

Open this **before** you curl, and leave the tab open.

Render

1. Browser: **https://dashboard.render.com** → sign in.
2. Click your **Web Service** (the one whose URL is `*.onrender.com`).
3. Open **Metrics**.
4. Find **CPU** and **Memory** (RAM) for that service. Ignore **Logs** for this exercise — logs are not a utilization number.
5. Note the graph's **timezone** vs your laptop clock. If they differ, write both in the notes.

Screenshot: Metrics page with CPU and memory visible. Filename like `render-cpu-mem-H-prep-154735.png`. If the dashboard shows a clock or a selected time range, include that in the shot.

Read the graph: free-tier Metrics are coarse (often ~1 minute). A flat line for a single `curl` is expected. You are learning **where the number lives**. In M2, a 60 s stage is what usually moves the needle. A screenshot of the stair-step window with your laptop time in `notes.md` is the hosted chain of evidence.

If you did not use Render: open whatever dashboard shows **instance CPU and RAM** for the public web process (Fly, Railway, a campus VM, …). Same job: timestamp, CPU, memory, screenshot. If the host has no CPU graph, write "API CPU not observable on this host" and lean on series **L** plus database stats — do not invent a percentage.

Do this now (hosted)

When	Laptop clock	Dashboard TZ if different	CPU (approx %)	Memory	Screenshot file
Idle (no curls for a minute)					
During / just after the **write** curl					

6.2 Local: start the API first

Series L only. Same commit as hosted. Example:

```
uvicorn main:app --host 127.0.0.1 --port 8000
```

Leave that terminal running. Open a **second** window for the monitor. You will identify **that API process**, not the browser, not a second Python that is only running `curl` or `load_baseline.py`.

not `pythnon` inside Linux. Measure **inside WSL** with `top` / `htop` (96.3 Linux). If the API is Windows `pythnon.exe`, use Task Manager.

6.3 Local: how to open the monitor

Do the OS that matches **where the API process runs**.

Windows — Task Manager (CPU and memory)

1. Press **Ctrl+Shift+Esc**. (Or Ctrl+Alt+Delete → Task Manager.)
2. If you see a tiny window, click **More details**.
3. Open the **Details** tab (not only **Processes** — Details is easier to sort).
4. Click the **CPU** column header so the busiest processes float to the top. Find `python.exe`, `pythonw.exe`, or `node.exe` (match how you start the API).
5. If **Memory** is missing: right-click a column header → **Select columns** → enable **CPU**, **Working set (memory)** or **Memory (active private working set)**.
6. Leave this window visible while you `curl` localhost.

Write: laptop clock, **process name**, CPU %, memory in MB.

Performance tab (left) is the **whole machine**. Use it only as a sanity check (laptop on fire). The M2 number is the **API process** on Details.

Windows — Resource Monitor (optional; disk and a second view of CPU)

1. Task Manager → **Performance** tab → **Open Resource Monitor** at the bottom.
   Or: Start menu → type `resmon` → **Resource Monitor**.
2. **CPU** tab: check the box next to your API process; the graphs filter to it.
3. **Memory** tab: same process; record **Working set** or **Commit**.
4. **Disk** tab: **Read (B/sec)** and **Write (B/sec)** for that process. If you cannot attribute disk to the API, write "disk not attributed."
5. **Network** tab: optional; useful if you want to see traffic to Supabase, not required today.

Resource Monitor is still a **gauge**. Same timestamped line as Task Manager; add disk only if the process is identifiable.

macOS — Activity Monitor

1. **Spotlight** (Cmd+Space) → type `Activity Monitor` → Enter.
   Or **Finder** → **Applications** → **Utilities** → **Activity Monitor**.
2. **CPU** tab. In the search box (top right), type `python` or `node` or `uvicorn`.
3. Select **your API process** (the one you started, not every Google Chrome Helper).
4. Record **% CPU** and, on the **Memory** tab, **Memory** (MB) for that same process.
5. **Disk** tab: **Bytes read/written** if you want I/O; optional today.
6. Menu **View** → **All Processes** if the process is missing. **View** → **Columns** can add PID.

Do not use the window's **CPU History** (whole Mac) as "API CPU."

Linux — GUI or terminal

GUI (Ubuntu and similar): Activities → type `System Monitor` (or `gnome-system-monitor`). **Processes** tab → find `python` / `uvicorn` / `node`. Record % CPU and Memory. **Resources** tab is the whole machine — same warning as Task Manager Performance.

Terminal (any distro, including WSL):

```
# find the API
ps aux | grep -E 'uvicorn|python|node' | grep -v grep
```

Watch that PID (`12345` is an example):

```
top -p 12345
```

Quit `top` with `q`. If `htop` is installed: `htop -p 12345`. Record **%CPU** and **RES** (resident memory) with a clock time.

Whole-machine disk (optional; not per-process unless you already know `pidstat`):

```
iostat -xz 1
```

Stop with Ctrl+C. If `iostat` is missing, skip disk and say so.

Docker: if the API is a container, in another terminal:

```
docker stats
```

Record container **CPU %** and **MEM USAGE**. That is the API box for series L.

6.4 Local: do this now

When	Laptop clock	Tool (Task Manager / Activity Monitor / top / docker stats)	Process or container name	CPU %	Memory
Idle					
During the localhost **read** or **write** curl					
~30 s after					

Write down: screenshot filename. Process name you watched: _____. One sentence: did you accidentally record Chrome or the load-generator Python instead of the API?

7. Resource signals — database (Postgres / Supabase)

The API is not the only box. Hosted Postgres is **not** in Task Manager. You fetch DB signals from the **Supabase dashboard** (or `psql` against local Postgres). You may **read** stats. Do not drop tables.

1. Browser: **https://supabase.com/dashboard** → your **project** (the one in `DATABASE_URL`).
2. **Graphs (if your plan shows them):** left sidebar → **Reports** (sometimes under **Observability**). Open anything named **database**, **CPU**, or **connections**. Screenshot with a timestamp, same rules as Render. Free tier may show little or nothing — then graphs are "not observable" and SQL still counts.
3. **SQL Editor** (always): left sidebar → **SQL Editor** → **New query**. Paste the statements below → **Run**. This is the connection count M2 expects even when Reports are empty.
4. Optional: **Database** (or **Settings** → **Database**) for max connections / pooler notes. Write the **max connections** number if you see it; that is the ceiling you might hit later.

Timezone: same as Render — compare dashboard time to the laptop clock.

If Postgres is not Supabase: use that host's CPU/connection graph if it exists, and still run `pg_stat_activity` in any SQL client (`psql`, DBeaver, pgAdmin).

7.2 How many connections are open?

```
SELECT count(*) AS connections
FROM pg_stat_activity;
```

```
SELECT state, count(*)
FROM pg_stat_activity
GROUP BY state
ORDER BY count(*) DESC;
```

`idle` vs `active` matters: a connection pool can hold many `idle` backends. A count of 1–8 at rest is common. A count that climbs toward the host **max connections** during load is a bottleneck candidate later.

```
SELECT pid,
       usename,
       application_name,
       client_addr,
       state,
       wait_event_type,
       query_start,
       left(query, 80) AS query_preview
FROM pg_stat_activity
WHERE datname IS NOT NULL
ORDER BY query_start DESC NULLS LAST;
```

You should recognize your API (or `psycopg` / SQLAlchemy) as `application_name` or in `query`. Dashboard traffic and the SQL editor itself also appear — that is why the count is never "exactly the load generator."

7.4 Optional: is the data growing?

```
SELECT relname AS table,
       n_live_tup AS approx_rows
FROM pg_stat_user_tables
ORDER BY n_live_tup DESC;
```

Run this **before** and **after** your write `curl`. If `approx_rows` moves on the table you inserted into, you have confirmed the write path hit Postgres. (`n_live_tup` is approximate; that is fine.)

If Postgres is on your laptop, run the same SQL in `psql`, pgAdmin, or DBeaver — not in Task Manager. Example:

```
psql "$DATABASE_URL" -c "SELECT count(*) FROM pg_stat_activity;"
```

CPU for a local `postgres` process can go in Activity Monitor / Task Manager / `top` the same way as the API; name the process `postgres`. Series **H** still needs Supabase (or your hosted DB), not this.

Do this now

Moment	Clock	`count(*)` from `pg_stat_activity`	`active` count	Notes
Idle				
Right after authenticated **read**				
Right after authenticated **write**				

Write down: did connections jump for one curl? (Often **no** — pooling.) That observation is useful: if M2 later shows high p99 and connections still flat, look at **CPU** or the **API event loop**, not "Postgres ran out of connections," unless the count actually rises.

---

8. Optional: 10-second driver smoke (still not M2)

Only if `login.py` / `load_baseline.py` are already in your repo and the instructor says to try them. **Concurrency 1**, **short duration**, **health or read**, then stop.

```
python load_baseline.py --help
```

Hosted health (no token):

```
python load_baseline.py --base https://YOUR.onrender.com --path /health --no-auth --concurrency 1 --warmup 2
--duration 10 --label prep-H-health-c1
```

Hosted read (token from the environment):

PowerShell: `--token $env:M2_TOKEN`
macOS/Linux: `--token "$M2_TOKEN"`

```
python load_baseline.py --base https://YOUR.onrender.com --path /api/messages --token YOUR_TOKEN --
concurrency 1 --warmup 2 --duration 10 --label prep-H-read-c1
```

Confirm a row appeared in `m2/results/summary.csv`. Then **stop**. Do not climb to 8 or 16 today.

9. Exit ticket (leave class with this filled)

A teammate who missed class should be able to copy this and hit your API.

```
Environment H BASE: _________________
Control:  GET _________________     auth? yes/no
Read:     ____ _________________     auth? yes/no
Write:    ____ _________________     JSON body: _________________
How we export the token (paste the command, not the token):
_________________
curl -w variables we will use for cold start and payload:
  time_total, size_download, http_code
Idle pg_stat_activity count: ____
Hosted API CPU: Render (or host) path we clicked: _________________
Local API CPU: tool + how we opened it: _________________
Process we watched locally (name): _________________
Windows users: we must type curl.exe  yes / not needed
```

If any line is blank, finish it before you start M2's 60-second stages.

- Not a capacity claim. One `curl` is not p99.
- Not permission to load-test Render in general or anyone else's homework.
- Not a reason to add Redis, a bigger instance, or a cache. Architecture stays: client → your API → PostgreSQL.

When the exit ticket is complete, you are ready for **M2 — Baseline Performance** and the **M2 Measurement Guide**.

Last modified: Wednesday, September 2, 2026, 11:27 AM

◀ M1 Build Guide — Render + Supabase (PDF)

Jump to activity

M2 - Measurement Guide ▶

UNIVERSITY of LOUISIANA LAFAYETTE

104 E. University Circle
Lafayette, LA 70503
(337) 482-1000

You are logged in as **Fupei Guo** (**Log out**)
Data retention summary
Get the mobile app